Solving the Travelling Salesman Problem

Design and Development Report

Authors to be completed

M1 AI, Université Grenoble Alpes, 2026

Abstract

This report describes the development of a C++ solver for the metric Travelling Salesman Problem. We cover the chronology of the work, the tools we relied on, the structure of the code and the reasons behind the main design choices. At the time of writing, the program reads both TSPLIB formats required by the specification, runs the trivial method and writes tours in the required format. A test suite of 84 cases covers the parser, the graph classes, the tour writer and the compiled executable.

1 Introduction

The goal of the project is a program that reads a TSP instance, computes a Hamiltonian cycle and writes it to a file. The specification asks for a build script at the root of the project, a `TSPsolver` executable taking an instance path, an output path and an optional method name, and a `tests` executable. It also fixes the input formats, namely a list of planar coordinates and the lower triangle of a weight matrix, and the output format, a permutation list ended by -1 and `EOF`.

Because the grading puts half of its weight on design quality, we separated the program into small components with clear responsibilities. Instance representation, distance computation, parsing, validation of tours and output are each handled by their own class, and each is covered by tests.

2 Development process

2.1 Setting up the project

We started on 30 September from the project template provided with the course, which already separates headers, sources and tests. We kept this layout. On 1 October we wrote the first version of the parser together with the code for the lower triangular matrix format, and created the empty files for the remaining parts of the architecture so that the structure was fixed before any algorithm existed.

The TSPLIB header needed some care. The entries may come in any order, the separator may or may not be preceded by a space across the provided files, and some entries such as comments can contain a second separator inside the value. We therefore split each header line at the first separator only, trim both parts, and ignore the entries we do not need.

2.2 Matrix instances and the first solver

By 10 October the parser handled the lower triangular format and the trivial method existed. Two properties of the provided files caused most of the work in this phase. First, the matrix is not guaranteed to be laid out with one row per line, so we read it as a stream of integers and rebuild the rows from the dimension. Second, one instance (`dantzig42`) is followed by a display data section holding coordinates that are only meant for drawing. Since the parser otherwise rejects data after the matrix, this section has to be recognised explicitly. We skip it up to the closing keyword and still reject any other trailing data.

2.3 Command line and coordinate instances

On 12 October we wrote the command line handling and connected the parser, the solver and the writer in `main`, so that the complete pipeline ran for the trivial method on the matrix instances. The build script was briefly lost during a clean up, and we restored it the same night, which showed us how easy it is to break the one requirement that is checked first. We then added the coordinate format in three small steps, the graph class, the Euclidean distance and the parsing of the coordinate section.

The coordinate files raised the question of identifiers. The specification describes cities by an index that starts at one, but nothing forces the lines to appear in increasing order. We store the coordinates at the position given by the identifier, after checking that it lies in the range of the dimension and has not been used before. The identifiers are also kept inside the instance so that the output file contains the identifiers of the input and not our internal zero based indices.

2.4 Review and tests

On 14 October we reviewed the code against the specification and wrote the test suite. The review found several defects that the manual runs of the previous days had not shown. We describe each one together with its cause and its correction.

Tests ignored by version control. Our ignore file contained the pattern `tests`, which was meant to hide the compiled test executable. A pattern without a path matches any file or directory with that name, so it also matched the `tests` source directory. Git therefore never tracked a single test file, and a clean checkout of the repository would have contained no tests. We removed the rule, since the build directory is already excluded and contains the executable.

Unchecked access in the matrix graph. The function that returns the weight of an edge read the matrix with an index that came directly from the caller. With an index beyond the dimension, the program read memory outside the vector, which is undefined behaviour and in our first test crashed the whole test run instead of reporting a failure. The function now compares both indices with the size and throws an out of range exception, which is what the coordinate graph already did.

Output file destroyed by an invalid tour. The tour writer opened the output file, which empties it, and only then checked that the tour was a valid permutation. A faulty solver would therefore have erased a previous result and left an invalid or empty file. The check now runs before the file is opened, so an invalid tour causes an error and leaves any existing file unchanged.

A test writes an invalid tour over a file with known content and verifies that the content is still there.

Inconsistent validation between the two graphs. The coordinate graph rejected an empty input but the matrix graph accepted an empty matrix, which would later give a graph with no vertices. The matrix graph now rejects it as well. In the same way, the coordinate parser ignored any text after the closing keyword while the matrix parser rejected it. We aligned the coordinate parser, so that both formats treat trailing data as an error.

Build output. The build script compiled into a subdirectory of the source folder and named the executable `main`, whereas the specification requires `build/TSPsolver` and `build/tests` at the project root. We corrected the names and the location, and the tests are now built by the same command.

2.5 Points that required a decision

Two of the provided instance files do not end with the `EOF` keyword, and the parser accepts this, since the keyword carries no information once the declared number of entries has been read. A more important issue concerns the edge weight types. The files `att48`, `burma14` and `ulysses16` declare a pseudo Euclidean distance and a geographical distance, while the specification only describes plain planar coordinates. We compute Euclidean distances for all coordinate files. Tour lengths on these three instances will therefore differ from the values published with TSPLIB, and they must not be compared with the published optima. We chose to document this limit and not to extend the parser, since the specification only requires the two data layouts and not every distance function of the library.

3 Tools and libraries

The solver is written in C++20 and uses only the standard library. The data structures needed here (vectors, strings, streams and smart pointers) are all available there, and avoiding other dependencies keeps the build simple on the machines used for grading. The project is built with CMake, and our own targets are compiled with warnings treated as errors, which keeps the code free of unused variables and sign comparison problems.

The tests use GoogleTest, which CMake downloads when the project is configured. It is the only external component, and only the test executable uses it. We chose it because it supports parameterised tests, which let us run the same checks on every provided instance. The instance generator in Python was provided by the course. We used Git for version control and Visual Studio Code on macOS.

4 Design and code structure

4.1 Overview

The code is organised as a static library that contains all the logic, and two executables that use it. The first executable, `TSPsolver`, contains only the entry point. The second, `tests`, links against the same library. This separation matters for testing, since no test needs to run the

program through its entry point to reach the parser or the solvers. Figure 1 shows the directories and the build targets.

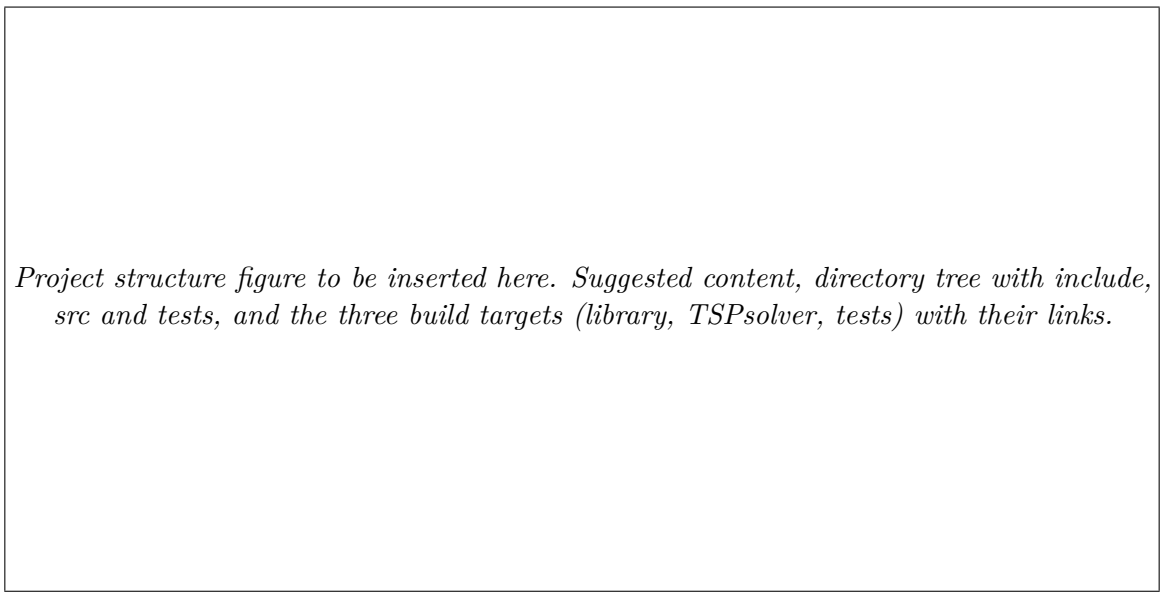


Figure 1. Directory layout of the project and the relation between the library, the solver executable and the test executable.

4.2 Representation of an instance

An instance is an object that combines a name, the list of original city identifiers and a graph. The graph is an abstract class with two operations, which return the number of vertices and the weight of an edge, plus a non virtual function that computes the length of a tour by summing the edges and closing the cycle. Placing the tour length in the base class guarantees that all graph types and all solvers evaluate a tour in the same way.

Two concrete graphs exist. The matrix graph stores the full symmetric matrix, which it builds from the lower triangle, and returns stored values. The coordinate graph stores only the points and computes each distance on demand. We prefer on demand computation for coordinate files because it needs memory proportional to the number of cities, while a matrix needs memory proportional to its square, which becomes relevant for the largest provided instance with 1002 cities. The cost is one square root per access, which is small compared with a cache miss in a large matrix. Because both classes hide their storage behind the same interface, a solver is written once and works with either format. The constructors validate their input, and the accessors check their indices, so that a defect in a solver produces an exception and not undefined behaviour.

Internally, vertices are numbered from zero, and the identifiers of the file are kept only in the instance. The conversion to the one based identifiers happens at a single place, the tour writer, which removes a whole class of off by one errors from the algorithms.

4.3 Parsing

The parser works in two stages. A header stage reads the key and value lines until it meets the keyword that opens a data section. It then checks that the name, the type and a positive dimension are present. A data stage is chosen from the keyword that ended the header, and it is

implemented in a separate source file for each format. The matrix stage additionally requires an explicit weight type and the lower diagonal row format, because other layouts would be read incorrectly if they were accepted.

The parser accepts a file path and any input stream. The stream version exists for the tests, which can then supply small malformed inputs as strings without creating files. Each call resets the internal state of the parser first, so one parser object can read several instances. All errors are reported with a runtime error exception whose message names the problem, and the entry point is the only place where these messages are shown to the user. We chose exceptions over return codes because a parsing failure cannot be corrected locally, and exceptions keep the successful path of the code readable.

The parser is strict about content that would change the meaning of the instance, such as negative weights, a non zero diagonal, duplicate identifiers, missing entries and trailing data, and tolerant about content that does not, such as blank lines, spacing around separators, Windows line endings, scientific notation and unknown header entries.

4.4 Solvers

A solver is any class that implements a common interface with a single function, which takes an instance and returns a tour represented as a vector of vertex indices. The trivial method returns the vertices in increasing order of their index. Because a solver only sees the graph interface and returns a plain vector, it does not depend on the input format or on the output format.

The entry point selects the solver from the value given with the `-m` option. The trivial method is the default, and an unknown name is reported as an error before any output file is created.

4.5 Writing tours

The tour writer converts a tour to the output format. Before writing, it checks that the tour is a permutation of all vertices, so that no solver can produce an output file that violates the specification. The output name is derived from the instance name by replacing a trailing extension, and only a trailing one. A second overload writes to any output stream, which is what the tests use to compare the exact text of the output with the example of the specification.

4.6 Entry point and error handling

The entry point reads the two positional arguments and the optional method flag, parses the instance, runs the selected solver and writes the tour. A missing argument, a method flag without a value and an unknown method each print a message and return a non zero status. Every error raised by the library, whether in parsing, solving or writing, is an exception derived from the standard exception class. The entry point catches it in one place, prints its message on the error stream and returns the same status. The library classes never print anything themselves, which keeps them reusable and lets the tests check errors without parsing text.

Figure 2 shows how these components are connected when the program runs.

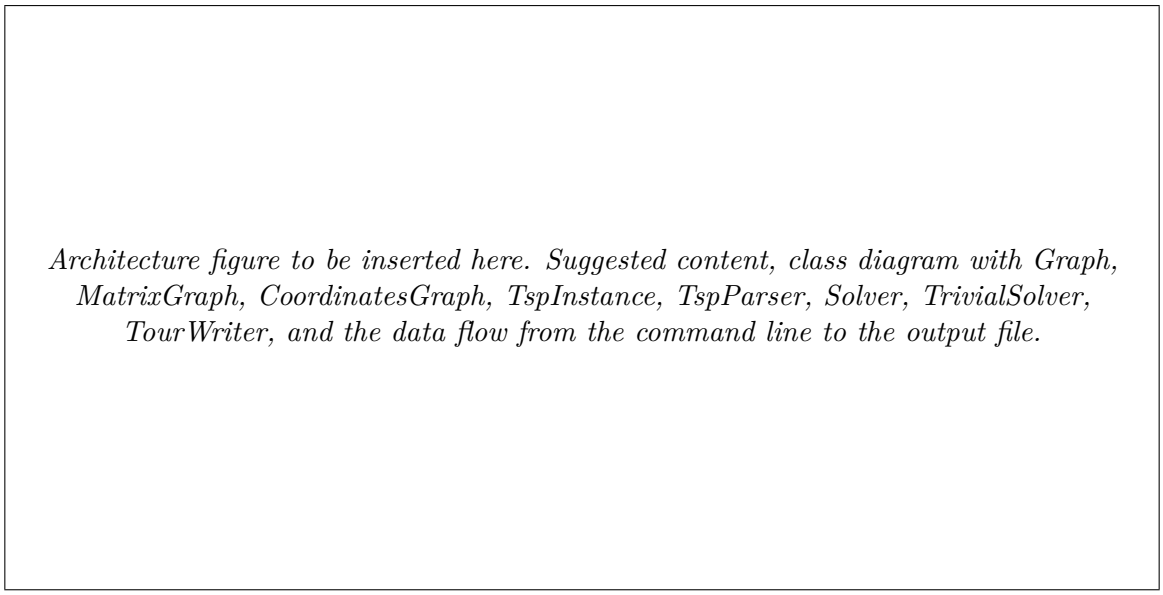


Figure 2. Main classes of the library and the data flow of a run, from the command line arguments to the tour file.

5 Testing

The test suite contains 84 cases in 11 groups, and it runs in well under a second. We organised it by component so that a failure points to a single class.

The graph tests check the Euclidean distance on a known triangle, symmetry, the zero diagonal, the triangle inequality on a sample, the closing edge of a tour, and the tour length of the example in the specification, which must equal 18. They also check that empty input and invalid indices are rejected. The instance tests cover the consistency checks of the constructor. The parser tests use both formats from the specification and many malformed variants. They include a missing or invalid dimension, a header line without a separator, truncated and surplus data, duplicate and out of range identifiers, negative and non integer weights, a matrix wrapped over several lines, a missing final keyword, and data after the final keyword. The tour writer tests compare the output with the example of the specification, check that identifiers and not indices are written, reject incomplete, repeated and out of range tours, and verify that failing to write leaves existing files untouched.

Two groups go beyond unit tests. A parameterised test loads each of the 15 provided instances, checks the dimension, the identifiers and the diagonal, runs the trivial method, and verifies that the result is a permutation with a positive length. A last group runs the compiled executable as a user would, with and without the method option, with an unknown method, a missing instance, an unwritable output and no arguments, which also covers the entry point. It also reads a produced tour back and checks that it contains every city exactly once.

Several tests failed before the corrections described in Section 2 and now pass, namely the empty matrix, the invalid matrix index, the protection of existing output files and the data after the final keyword in a coordinate file. We therefore consider them regression tests for real defects and not only coverage.

6 Limitations

The weight types discussed in Section 2 are not supported. Euclidean distance is used for every coordinate file, without the rounding that TSPLIB applies to some types, so the tour lengths we compute can differ slightly from published values. The trivial method makes no attempt to produce a short tour, since its purpose is to validate the pipeline from parsing to output.

7 Reflections

The most useful decision of the first weeks was to fix the structure of the code early. It allowed us to add the second input format in three small commits without touching existing code. The review of 14 October was less comfortable, since it showed that several parts we considered complete had behaviour that we had never exercised. The problems with the output file and the matrix accessor would not have appeared in normal use, and we only found them because the tests supplied inputs that a careful user would not. If we repeated the project, we would write the tests of each component at the same time as the component.

8 Use of AI tools